

A parallel multiscale FIM approach in solving the Eikonal equation on GPU

Jingqi Zhang^a, Zihao Zhou^a, Lixin Ren^b, Junyuan Liu^a, Ying Li^a, Xiaowei He^b^{*}

^a University of Chinese Academy of Sciences, China

^b Institute of Software, Chinese Academy of Sciences, China

ARTICLE INFO

Keywords:

Fast iterative method
Multiscale technique
Signed distance field
Eikonal equation
Parallel algorithms

ABSTRACT

Signed Distance Fields (SDFs) are essential in various applications, particularly in level set problems, where computing the SDF is equivalent to solving the Eikonal equation. Common approaches to solving these equations include the Fast Marching Method (FMM), the Fast Sweeping Method (FSM), and the Fast Iterative Method (FIM). However, FMM and FSM face significant challenges in parallelization, increasing interest in developing FIM for GPU architectures. In this paper, we extend the innovative FIM algorithm (Huang, 2021), which is GPU-friendly but relies on a single uniform grid, by incorporating multiscale techniques to accelerate wavefront propagation from source points to infinity. Unlike the traditional Fast Iterative Method, which operates on a single uniform grid and propagates the wavefront at a constant speed of one grid spacing per iteration, our multiscale approach applies a hierarchy of varying propagation speeds to accelerate the convergence. Once all source and infinite points are properly initialized, only a few FIM iterations are required to refine the values of points near the source. A coarser-grained scale, with twice the spacing of the finer grid, is then used to propagate values from accepted and tentative points to the outer regions. This process is repeated until the top-level scale is reached. Subsequently, we reverse this process by performing FIM calculations from the coarsest scale until reaching the finest grid, thereby completing a V-cycle. With multiscale V-cycles, the solution progressively converges across the entire computational domain. Comparative experimental results show that our algorithm improves computational efficiency by approximately 128% over the GPU-based Fast Marching Method and by a factor of 23 compared to the improved FIM algorithm (Huang, 2021) at scale of $N = 2E8$. This optimized approach applies to numerical simulations of multi-body systems, including fluid–structure interactions, as well as numerical analyses of flooding and earthquake scenarios.

1. Introduction

In Earth science, the simulation and prediction of natural disasters, such as earthquakes and floods, are essential for safeguarding human lives and property and for developing effective disaster management strategies. In flood disaster simulation, the complex dynamics of floodwaters on land involve a continuously shifting boundary between inundated areas and terrestrial terrain. Rising water levels in rivers, lakes, and other bodies of water result in the flooding of adjacent urban areas, agricultural lands, and natural ecosystems. Additionally, variations in topography and geomorphology – such as mountains, plains, and valleys – play a crucial role in determining flow patterns, velocities, and the extent of flood propagation.

By defining the dynamic boundary as a zero-level set [1–4], the level set function effectively captures the signed distance from each point in the simulation domain to the flood boundary, leveraging its inherent signed distance property. This representation not only visually illustrates the real-time extent of flooding but also provides crucial

quantitative insights for forecasting the direction and velocity of flood diffusion. In urban flood simulations, the application of the level set function facilitates the identification of flood-prone areas and buildings at risk of inundation. This information is essential for devising urban flood prevention strategies and formulating emergency evacuation plans.

In addition to flooding, seismic simulations encompass events such as fault movements and strata displacements that alter geological structures, resulting in dynamic changes in the physical properties of subsurface media. This complexity presents challenges for simulating seismic wave propagation. The level set function proves advantageous in defining the seismic wave interface within the subsurface medium and capturing the formation of new interfaces arising from seismic-induced structural alterations.

During numerical simulations, the signed distance property of the level set function may not be automatically preserved due to factors such as numerical approximation and iterative computations involved

* Corresponding author.

E-mail address: xiaowei@iscas.ac.cn (X. He).

<https://doi.org/10.1016/j.cad.2025.103949>

Received 10 June 2025; Accepted 11 August 2025

Available online 2 September 2025

0010-4485/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

in solving the relevant equations [5]. Consequently, periodic restoration of this property is necessary. The core of recovering the signed distance lies in solving the following Eikonal equation [6].

$$H(x, \nabla \phi) = |\nabla \phi(x)|^2 - \frac{1}{f^2(x)} = 0, \quad (1)$$

where $x \in \Omega$ represents the position in the computational domain Ω , the function $\phi(x)$ represents the location of the interface, and $f(x)$ is the speed function.

In solving the above equation, three approaches are commonly used, including the Fast Marching Method (FMM), the Fast Sweeping Method (FSM), and the Fast Iterative Method (FIM). However, both FMM and FSM have limitations in terms of parallelism [7]. Specifically, FMM struggles with efficient parallel computation due to its reliance on priority queues (heap structures) to maintain the order of node updates, leading to irregular memory access patterns and frequent heap operations. Similarly, FSM is constrained by a fixed scanning order (e.g., multi-directional alternating traversal), which creates strong data dependencies and requires the serial execution of different scanning phases. Consequently, research on parallel implementations of FIM has attracted significant attention.

Existing studies have made progress in enhancing the accuracy of FIM (e.g., Cai's second-order difference [8]) and improving its parallel efficiency (e.g., Huang's process splitting [9]). However, challenges remain when addressing complex boundary and multi-scale problems. Rigid fixed grids struggle to balance global efficiency and local accuracy, as dual grids are only locally refined at the source point and cannot dynamically adapt to regions where multiple wavefronts intersect. In seismic wave simulations, for example, wavefronts from different earthquake sources or in acoustic models of concert halls where sound waves from various origins converge, create intricate interference patterns. These scenarios alter wave propagation and physical field distributions, testing the accuracy and efficiency of numerical algorithms. Moreover, structured uniform grids are poorly suited for non-uniform or anomalous geometries, leading to redundant computations and inefficient storage.

Motivated by the need to balance global efficiency and local accuracy, this paper extends the FIM algorithm [9], which relies on a single uniform grid, by introducing hierarchical grid levels and developing a novel multiscale technique to accelerate convergence in solving the Eikonal equation. This paper first reviews previous work on the FMM, FSM, and FIM algorithms in Section 2. Sections 3 and 4 detail the design of the improved FIM algorithm, which incorporates parallel development and a multiscale technique. Section 5 demonstrates the algorithm's efficiency improvements through experimental setups and result analysis. Finally, Section 6 presents conclusions and discusses future research directions.

2. Related work

FMM [10], FSM [11], and FIM [12] are all employed to solve Eikonal equations [13]. Each method possesses distinct characteristics in terms of solving approaches, time complexity, and relative advantages and disadvantages, thereby offering a range of options for numerical solutions to this equation.

Existing research has predominantly focused on achieving a balance between algorithmic parallelism and accuracy [14], yet the optimization potential of FIM remains largely untapped. In addressing the multigrid technology gap, Chen et al. (2015) [15] demonstrated the efficacy of applying multigrids in FSMs, suggesting that similar strategies could be adapted for FIM to reduce redundancy and enhance adaptability for multi-scale problems.

2.1. Fast marching method (FMM)

The Fast Marching Method proposed by Sethian (1996) [16] guarantees the monotonicity of the solution through a narrow-band expansion strategy with a time complexity of $O(N \log N)$, N being the total number of grid points, and $\log N$ originating from the overhead of sorting the Tentative set. The Fast Marching Method (FMM), proposed by Sethian (1996) [16], ensures the monotonicity of the solution through a narrow-band expansion strategy, achieving a time complexity of $O(N \log N)$, where N represents the total number of grid points, and the $\log N$ term arises from the overhead associated with sorting the Tentative set.

The fundamental approach of the FMM involves using a stack to sort the points on the advancing wavefront based on the properties of the solution, thereby ensuring that the feature line represents the fastest descending direction of ϕ . The FMM categorizes the computational area into three regions: accepted, tentative, and distant. Initially, the boundary conditions are placed into the Accepted set, while the Tentative set is initialized with points whose neighbors are not included in the Accepted set. The values of these points are computed, and the tentative set is structured as a minimum heap. In each iteration, points are extracted from the top of the heap and added to the Accepted set. Subsequently, the neighbor values are updated, and any qualifying neighbors are moved from the Distant set to the Tentative set until the Tentative set is empty.

Although the FMM is significantly faster than other numerical methods in achieving high accuracy, it remains a critical component of contemporary level set method implementations [17]. However, its stringent data dependencies result in poor parallel scalability; for instance, Yang and Stern (2017) [18] reported less than 15% efficient scaling with 10,000-core parallelism. Chacon and Vladimirovsky (2012) [19] proposed a two-scale hybrid method that combines the advantages of the coarse mesh from the FMM with the fine mesh of the FSM, yet high-dimensional scalability continues to be limited. Although numerous studies have attempted to parallelize the FMM through coarse-grained parallelization via domain decomposition, improvements in computational efficiency have been minimal [20].

2.2. Fast Sweeping Method (FSM)

The Fast Sweeping Method, developed by Zhao (2004) [21], is based on multi-directional Gauss–Seidel generation selection, offering a low memory footprint for structured grids. The fundamental approach of the Fast Sweeping Method relies on Godunov's windward differential format discretization and employs Gauss–Seidel iterations along with alternating sweeping to solve the discrete system. In a two-dimensional grid, at least 2^n directional scans (one for each quadrant) are required, with the alternating scan order for points with unfixed values determined by the windward differential format until convergence is achieved. The time complexity of the Fast Sweeping Method is $O(N)$, where N denotes the total number of grid points. Detrixhe et al. (2013) [22] proposed a parallel implementation of the Fast Sweeping Method (FSM) based on Cuthill–McKee ordering, which maximizes thread utilization through the ordered distribution of grid points. This parallel approach demonstrates minimal decay in efficiency under ultra-multi-threaded scenarios and maintains a comparable number of iterative convergences to traditional FSMs (e.g., Zhao 2007 [23]). However, it is sensitive to the distribution of initial conditions and does not sufficiently address load balancing for highly unstructured grids. Shrestha and Senocak (2018) [11] proposed a distributed parallel strategy also based on Cuthill–McKee ordering, but efficiency declines significantly in complex boundary scenarios. Chen et al. (2015) [15] introduced the Lax–Friedrichs multigrid method to enhance computational efficiency, yet issues related to cache utilization due to multi-directional scanning continue to limit GPU performance.

2.3. Fast Iterative Method (FIM)

The Fast Iterative Method (FIM) proposed by Jeong and Whitaker (2008) [24] employs dynamic active queues to facilitate asynchronous updates, making it particularly well-suited for parallel architectures due to its timing independence. The methodology of the FIM involves maintaining a narrow band (active list) that contains the updated grid points. During initialization, the initial values of grid points in the designated area are set according to the boundary conditions, marking them as source points, while all other points are initialized to infinity. The neighbors of the source points are then added to the active list. When updating, the values in the active list are refreshed based on the neighboring source points. Once a point converges, it is added to the source point set, and the algorithm checks for convergence in adjacent points. Neighbors that remain unconverged or are set to infinity are added to the active list until it is empty.

Hong and Jeong (2016) [25] introduced a dynamic adaptive region decomposition method for multi-GPU systems to enhance the parallel efficiency of BlockFIM (Block-based Fast Iterative Method), which serves as an Eikonal equation solver. Subsequent developments, such as GO-FIM (Hong and Jeong, 2017) [26], have aimed to reduce redundancy through block-level causality; however, challenges related to multi-GPU load balancing persist. Huang (2021) [9] improved the convergence-checking mechanism of FIM by implementing selective update points and error correction queues, achieving performance enhancements of 5 to 28 times faster than conventional algorithms on GPUs. Nonetheless, FIM continues to suffer from a high redundant computation rate (up to 40%) and increased memory requirements for second-order differencing [8], which represent significant drawbacks.

In the realm of hybrid parallel strategies, Detrixhe and Gibou (2016) [27] combined dual-granularity task pooling of coarse-grained (CPU) and fine-grained (GPU) tasks to achieve heterogeneous architectural load balancing, attaining 86% utilization. Chacon (2012) [19] proposed a two-scalar hybrid approach that combines coarse-grained FMM with fine-grained FSM, resulting in a 50% reduction in arithmetic demand; however, this method is prone to artifacts when scaling in three dimensions.

Existing research has predominantly focused on achieving a balance between algorithmic parallelism and accuracy [14], yet the optimization potential of FIM remains largely untapped. In addressing the multigrid technology gap, Chen et al. (2015) [15] demonstrated the efficacy of applying multigrids in FSMs, suggesting that similar strategies could be adapted for FIM to reduce redundancy and enhance adaptability for multi-scale problems.

3. Fundamentals of the fast iterative method

3.1. Godunov upwind discretization

For the three-dimensional case, Eq. (1) can be discretized on a 3D Cartesian grid as follows [24]:

$$g_{i,j,k} = \left[\frac{(\phi_{i,j,k} - \phi_{i,j,k}^{xmin})^+}{h} \right]^2 + \left[\frac{(\phi_{i,j,k} - \phi_{i,j,k}^{ymin})^+}{h} \right]^2 + \left[\frac{(\phi_{i,j,k} - \phi_{i,j,k}^{zmin})^+}{h} \right]^2 = \frac{1}{f_{i,j,k}^2} \quad (2)$$

where h represents the grid spacing. The discrete approximation of travel time at $\mathbf{x}$ can be obtained by solving Eq. (2), where $g_{i,j,k}$ represents the discrete approximation of the travel time at grid node $\mathbf{x} = (x_i, y_j, z_k)$, $(x)^+ = \max(x, 0)$ [9]. $\phi_{i,j,k}^{xmin}$, $\phi_{i,j,k}^{ymin}$ and $\phi_{i,j,k}^{zmin}$ are defined as

$$\begin{aligned} \phi_{i,j,k}^{xmin} &= \min(\phi_{i-1,j,k}, \phi_{i+1,j,k}) \\ \phi_{i,j,k}^{ymin} &= \min(\phi_{i,j-\alpha,k}, \phi_{i,j+1,k}) \\ \phi_{i,j,k}^{zmin} &= \min(\phi_{i,j,k-1}, \phi_{i,j,k+1}) \end{aligned} \quad (3)$$

3.2. Quadratic solver [9]

In FIM, Eq. (2) is typically solved using a quadratic solver. The formula in Eq. (4), used to compute the solution to the 3D Eikonal equation, takes as input the values of a_i ($i = 1, 2, 3$) from upwind neighbors. These upwind neighbors are defined as the grid points in the direction opposite to the flow of information propagation (analogous to the “upstream” direction in fluid dynamics). Here, a_i represents the precomputed solution values (e.g., travel time or distance metrics) from these upwind nodes, sampled along the principal grid directions, with the condition $a_1 \leq a_2 \leq a_3$. In this formula, f is treated as the speed:

$$\phi_{i,j,k} = \begin{cases} \frac{1}{6} [2\sum_{i=1}^3 a_i + \sqrt{4(\sum_{i=1}^3 a_i)^2 - 12(\sum_{i=1}^3 a_i^2 - \frac{h^2}{f_{i,j,k}^2})}], & |a_1 - a_3| < h \\ \frac{1}{2} [a_1 + a_2 + \sqrt{2\frac{h^2}{f_{i,j,k}^2} - (a_1 - a_2)^2}], & |a_1 - a_2| < h \\ a_1 + \frac{h}{f_{i,j,k}}, & \text{otherwise} \end{cases} \quad (4)$$

Given above formulation, the innovative FIM algorithm [9] is GPU-friendly; however, it is constrained by its reliance on a single uniform grid, which limits the speed of wavefront propagation.

3.3. Motivation

Our objective is to enhance wavefront propagation by incorporating multiscale techniques. A natural approach is to apply a hierarchy of uniform grids with varying resolutions, constructed from the finest grid and progressively merged to form coarser ones. A standard multigrid method can then be employed to accelerate convergence by reducing errors at multiple scales. However, the frequent restriction and interpolation operations required by the multigrid method can complicate its implementation and may also reduce the accuracy in solving the Eikonal equation. Since using a coarser grid is equivalent to employing a larger wavelength in wavefront propagation, our multiscale approach remains grounded in a single uniform grid. However, we utilize a hierarchy of wavelengths when solving the Eikonal equation. By defining a sequence of wavelengths to be $\alpha = 1, 2, \dots, 2^N$, the solution of ϕ at a given scale α can be derived as

$$\phi_{i,j,k}^\alpha = \begin{cases} \frac{1}{6} [2\sum_{i=1}^3 a_i + \sqrt{4(\sum_{i=1}^3 a_i)^2 - 12(\sum_{i=1}^3 a_i^2 - \frac{(\alpha h)^2}{f_{i,j,k}^2})}], & |a_1 - a_3| < \alpha h \\ \frac{1}{2} [a_1 + a_2 + \sqrt{2\frac{(\alpha h)^2}{f_{i,j,k}^2} - (a_1 - a_2)^2}], & |a_1 - a_2| < \alpha h \\ a_1 + \frac{\alpha h}{f_{i,j,k}}, & \text{otherwise} \end{cases} \quad (5)$$

where a_1, a_2, a_3 represent either the ascending or descending order of $\phi_{i,j,k}^{\alpha,xmin}, \phi_{i,j,k}^{\alpha,ymin}, \phi_{i,j,k}^{\alpha,zmin}$

$$\begin{aligned} \phi_{i,j,k}^{\alpha,xmin} &= \min(\phi_{i-\alpha,j,k}, \phi_{i+\alpha,j,k}) \\ \phi_{i,j,k}^{\alpha,ymin} &= \min(\phi_{i,j-\alpha,k}, \phi_{i,j+\alpha,k}) \\ \phi_{i,j,k}^{\alpha,zmin} &= \min(\phi_{i,j,k-\alpha}, \phi_{i,j,k+\alpha}) \end{aligned} \quad (6)$$

In the next section, we will show how to use a combination of different scales to accelerate the convergence in solving the Eikonal Equation.

4. Multiscale fast iterative method

We extend the innovative Fast Iterative Method (FIM) algorithm [9], which is GPU-friendly but limited to a single uniform grid, by incorporating multigrid techniques to accelerate wavefront propagation from source points to infinity. Unlike the traditional FIM, which operates on a uniform grid and propagates the wavefront by one grid spacing per

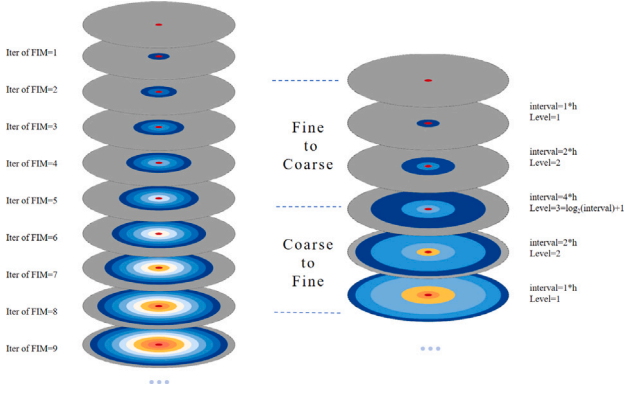


Fig. 1. The process of the FIM algorithm [9] and our multiscale FIM algorithm.

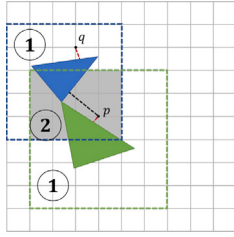


Fig. 2. Initialization from a Triangular Mesh.

iteration, our multiscale approach utilizes a hierarchical wavelengths to speed up convergence. Fig. 1 provides a high-level illustration of the multiscale approach, demonstrating significantly faster wavefront propagation compared to the standard FIM algorithm [9] (here after referred to as Tra.FIM). The main algorithm consists of two parts, the initialization step and the V-cycle update step.

4.1. Initialization

The initialization process in this paper aligns with the traditional FIM approach. Points within the computational domain are categorized as accepted, infinite, or tentative. Specifically, points in the Accepted region are assumed to have fixed values; points in the Tentative region are those in the Active set whose values are waiting to be updated; points in the Infinite region have infinite values and will not be updated until added into the Active set. At the beginning, a narrow band of accepted points should be maintained. Depending on the input type, two initialization strategies are employed in this work.

Initialization from a triangular mesh. As shown in Fig. 2, When solving the signed distance field (SDF) using triangular meshes, an axis-aligned bounding box (AABB, represented by the blue and green dashed frames in the figure) is first constructed for each triangular facet to quickly filter out the facets that may affect grid points. Atomic operations are employed to record the triangular facets associated with each grid point (grid points in the overlapping gray area are associated with multiple facets). Subsequently, for each grid point, the shortest distance from the point to each associated triangular facet is computed (e.g., grid point q computes the distance to the blue facet, while point p compares distances to both blue and green facets). The minimum value is taken as the SDF for that grid point. The sign of the distance (negative for interior points, positive for exterior points) is determined based on the positional relationship between the point and the facet. Through multiscale optimization refinement, the SDF, where each grid point represents the "signed distance to the nearest triangular facet", is ultimately obtained, achieving the construction of the grid-based distance field from triangular meshes.

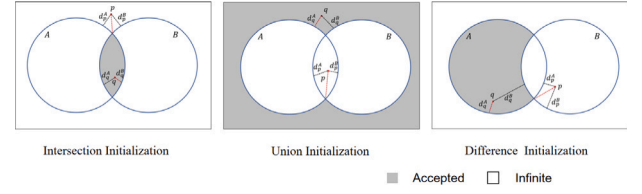


Fig. 3. Initialization from a Boolean of Two-Level Sets.

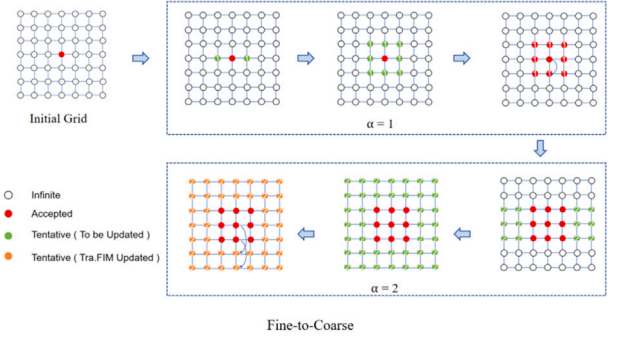


Fig. 4. This figure shows the computational process of our multiscale FIM algorithm with two scales including $\alpha = 1$ and $\alpha = 2$.

Initialization from a boolean of two-level sets.

$$\begin{aligned} \phi_3 &= \phi_1 \& \phi_2 = \max(\phi_1, \phi_2), & \text{intersection} \\ \phi_3 &= \phi_1 | \phi_2 = \min(\phi_1, \phi_2), & \text{union} \\ \phi_3 &= \phi_1 \setminus \phi_2 = \max(\phi_1, -\phi_2), & \text{difference} \end{aligned} \quad (7)$$

Boolean operation initialization leverages Signed Distance Functions (SDFs, with negative values for interior regions and positive values for exteriors) and logical operations. It marks regions as "Accepted" or "Infinite", establishing a geometric-topological "seed state" for level-set iterations (Fig. 3). Based on the intersection operation in Eq. (7), only regions inside (or on the boundary of, with non-positive SDF) both shapes are marked as Accepted, and the rest as Infinite, corresponding to the overlapping gray area of two circles. For union, following the union operation in Eq. (7), regions outside (or on the boundary of, with non-negative SDF) both shape are marked as Accepted (corresponding to the overall gray area outside two circles), and areas inside either as Infinite. For difference, following the difference operation in Eq. (7), regions inside A (with negative ϕ_1) and outside B (or on the boundary, with non-positive $-\phi_2$) are marked Accepted, corresponding to the gray area of A excluding its overlap with B, while the areas inside B or outside both shapes are infinite. Through SDF-based logical operations, they support subsequent iterations for accurate level-set function computation.

4.2. V-cycle update

After initializing the narrow band of Accepted points, the next step is to propagate the wavefront and assign appropriate values to the Infinite points. The core idea of our multiscale FIM is to progressively increase the wavelength α from 1 to 2^N to accelerate wavefront propagation. However, if the wavelength becomes excessively large, the solution for ϕ may yield incorrect results. This inaccuracy arises from erroneous intersections between the narrow band of Accepted points and the neighboring points used in computing $\phi_{i,j,k}^\alpha$. In other words, the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ at each grid point should be constrained by the initial narrow band of Accepted points. The entire multiscale FIM process consists of two phases: a fine-to-coarse phase and a coarse-to-fine phase, as demonstrated in Fig. 1.

Fine-to-coarse phase. At the beginning, the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ for all grid nodes are initialized to zero. The algorithm always starts with the finest scale using a wavelength of 1. The upper part of Fig. 4 depicts the whole procedure. First, a tentative check is performed along each axis. Taking the tentative check along the x -axis for example, we will check the status for any two neighboring grid points. In case one of the grid point belongs to the Infinite region while the other belongs to either the Accepted region or the Tentative region, we will add the grid point in the Infinite region to the Tentative region. Beside, the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ for the newly added Tentative point is set to 1. After the tentative check along the x -axis is finished, a similar check should be taken along the y and z axis, respectively, which completes the tentative check for a wavelength of 1. Then, Eq. (5) is applied to update the values for all grid points inside the Tentative region in parallel. Notice all newly added grid points in the Tentative are assigned with a value of $\hat{\alpha}_{i,j,k} = 1$. We continue this process by increasing the wavelength from 1 to 2^N with 2^N represents the lower bound of half of the maximum resolution of the finest grid, i.e., $2^N \geq \frac{1}{2} \max(N_x, N_y, N_z)$. For each newly added grid point, the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ is assigned to the value of the corresponding wavelength. After the wavelength is reached to 2^N , all grid points in the Infinite region are finally added to the Tentative region. Besides, the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ stored on each grid point represents a conservative estimation of its distance to the boundary.

Coarse-to-fine phase. The coarse-to-fine procedure begins with an initial wavelength of $\alpha = 2^N$ and gradually decreases to 1. The value of ϕ at each grid point is iteratively refined using ϕ^α . During this process, two special cases must be carefully handled. The first is to avoid interference with the initial Accepted region. Given the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$, we compare the current wavelength α with $\hat{\alpha}_{i,j,k}$. If $\alpha > \hat{\alpha}_{i,j,k}$, indicating that the neighbors used to compute $\phi^\alpha_{i,j,k}$ may intersect with the initial Accepted region, then $\phi_{i,j,k}$ should not be updated. Otherwise, $\phi_{i,j,k}$ is updated to $\phi^\alpha_{i,j,k}$. The second case is to avoid exceeding the grid boundary when computing $\phi^\alpha_{i,j,k}$ using $\phi^{\alpha,xmin}_{i,j,k}$, $\phi^{\alpha,ymin}_{i,j,k}$, and $\phi^{\alpha,zmin}_{i,j,k}$ as defined in Eq. (6). For example, when computing $\phi^{\alpha,xmin}_{i,j,k}$, if both $(i - \alpha, j, k)$ and $(i + \alpha, j, k)$ lie outside the computational domain, we set $\phi^{\alpha,xmin}_{i,j,k} = \infty$. If only one of the neighbors is outside the domain, we assign the value of the valid neighbor to $\phi^{\alpha,xmin}_{i,j,k}$.

The two procedures described above together form a V-cycle, which is repeated until either the Tentative region becomes empty or a user-defined threshold is reached. A minor distinction is that the maximum permissible wavelength $\hat{\alpha}_{i,j,k}$ for each grid point is computed only during the first V-cycle and remains unchanged in subsequent cycles. Furthermore, if the difference between $\phi_{i,j,k}$ and the updated value $\phi^\alpha_{i,j,k}$ is smaller than a specified threshold, the grid point (i, j, k) is added to the Accepted region to enhance computational efficiency. Refer to Algorithm 1 for more details.

5. Results

For the numerical experiments, we implemented parallel versions of the Fast Sweeping Method [23], the Fast Iteration Method [9], and the Improved Fast Iteration Method (the Fast Iteration Method combined with multiscale techniques) using NVIDIA GeForce RTX 3060 GPUs to demonstrate the efficiency of the parallel algorithms.

Initially, this study conducts comparative experiments in four different scales of three-dimensional computational space to illustrate the convergence of the Improved Fast Iteration Method and to assess the limited impact of the initial values on experimental accuracy.

Subsequently, we compare the performance of various algorithms across these four computational scales, focusing on convergence speed and efficiency while maintaining the same accuracy threshold (RMSE < 0.05). The results demonstrate that the fast iterative method combined

Algorithm 1 Parallel Multiscale FIM

Require: V_{Cycle}, N ;

- 1: Do initialization according to Section 4.1;
- 2: Set $\hat{\alpha}_{i,j,k} = 0$ for all grid nodes;
- 3: **for** $v = 0$ to V_{Cycle} **do** ▷ fine to coarse
- 4: **for** $exp = 0$ to N **do**
- 5: $\alpha = 2^{exp}$;
- 6: **if** $v = 0$ **then**
- 7: Tentative check along the x, y and z axis;
- 8: **if** (i, j, k) has one neighbor as Infinite and the other not **then**
- 9: Add (i, j, k) to Tentative region;
- 10: $\hat{\alpha}_{i,j,k} = \alpha$;
- 11: **end if**
- 12: **end if**
- 13: **if** (i, j, k) belongs to the Tentative region **then**
- 14: **if** $\alpha \leq \hat{\alpha}_{i,j,k}$ **then**
- 15: Update $\phi_{i,j,k}$ according to Eq.(5);
- 16: **if** $\|\phi^\alpha_{i,j,k} - \phi_{i,j,k}\| < \epsilon$ **then**
- 17: Add (i, j, k) to Accepted region;
- 18: **end if**
- 19: **end if**
- 20: **end if**
- 21: **end for**
- 22: **for** $exp = N$ to 0 **do** ▷ coarse to fine
- 23: $\alpha = 2^{exp}$;
- 24: **if** (i, j, k) belongs to the Tentative region **then**
- 25: **if** $\alpha \leq \hat{\alpha}_{i,j,k}$ **then**
- 26: Update $\phi_{i,j,k}$ according to Eq.(5);
- 27: **if** $\|\phi^\alpha_{i,j,k} - \phi_{i,j,k}\| < \epsilon$ **then**
- 28: Add (i, j, k) to Accepted region;
- 29: **end if**
- 30: **end if**
- 31: **end if**
- 32: **end for**
- 33: **end for**

with multiscale techniques achieves global solution convergence more rapidly within large-scale three-dimensional spatial computational domains.

Finally, this paper applies the aforementioned algorithms to the dynamic calculations of multi-body systems. By designing multi-body systems with varying complexities, we establish that all algorithms operate stably to meet the high-frequency demands necessary for quickly and accurately computing the signed distances of each geometrical entity.

5.1. Convergence and initial value dependence of FIM with multiscale

This paper presents a comparative experiment in four different scales of three-dimensional computational space to demonstrate the convergence of the Improved Fast Iterative Method. Additionally, it establishes that the experimental results are independent of the initial values.

In this context, the computation of Eq. (1) is performed using a simple sphere as the initial boundary Γ and a constant velocity function $f(x) = 1$. This approach aims to solve a signed distance field within a three-dimensional computational space. Fig. 5 provides a visual representation of the computational solution for ϕ by intercepting the plane at the center of the sphere.

To illustrate the convergence speed, this paper introduces the Root Mean Square Error (RMSE) defined in Eq. (8). Here, y_i represents the exact signed distance value corresponding to the grid points and $\hat{y}_i$ denotes the corresponding approximate signed distance value. The

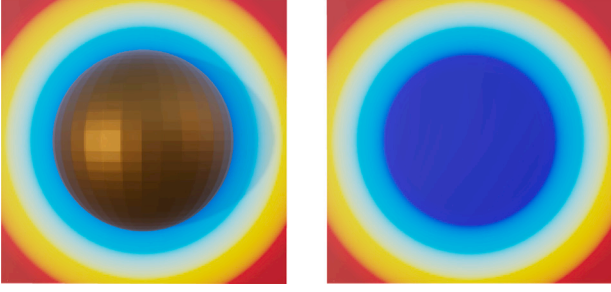


Fig. 5. The figure shows the solution of Eq. (1) for $f(x) = 1$ and intercepts the computational solution in the plane where the center of the sphere is located. Here the initial condition is a sphere of radius $X = 1.35$. (Since exactly the same visualization is obtained at different scales, only one figure is shown here.)

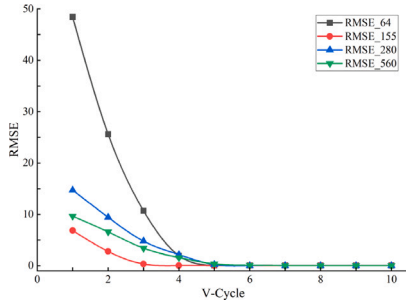


Fig. 6. The figure shows the variation of RMSE values corresponding to the iterative process of the V-cycle at four scales. Specifically, calculations are performed for $f(x) = 1$ in Eq. (1) at four different scales of 3-dimensional space and plotted against the RMSE (Eq. (8)) and the number of V-cycles.

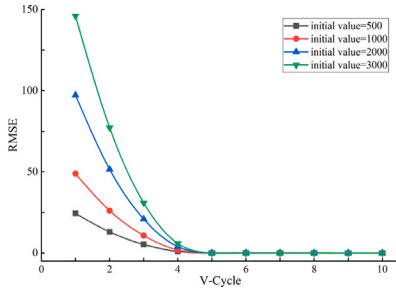


Fig. 7. The figure shows the variation of RMSE values with increasing numbers of V-cycles for initial values of 500, 1000, 2000, and 3000, respectively, at a $64 \times 64 \times 64$ scale.

variable m indicates the number of points in the computational space. For instance, in a computational scale of $64 \times 64 \times 64$, m is calculated as $64 \times 64 \times 64 = 262,144$. This metric is employed to assess the accuracy of the computed values.

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2} \quad (8)$$

The corresponding RMSE values for the number of V-cycles at four different scales are presented in Table 3 and Fig. 6 (with an initial point value of 1000.0). As observed, in three-dimensional space across varying computational scales, the RMSE gradually decreases toward zero as the number of V-cycles increases. This indicates that the solution progressively converges with an increasing number of V-cycles.

To investigate whether convergence is influenced by the initial value, the variation of RMSE values across four initial values is plotted.

As shown in Table 4 and Fig. 7, all curves exhibit a similar trend for initial values of 500, 1000, 2000, and 3000. Additionally, different initial values have only a minor effect on the RMSE values in the early stages. However, as the number of V-cycles increases, the RMSE values gradually converge. Therefore, it can be concluded that changes in the initial value do not affect the convergence behavior of the algorithm.

5.2. Comparison with FSM [23] and traditional FIM [9] (basic model)

This section focuses on signed distance field solving and revolves around the fast iterative class of algorithms. For different geometric initial conditions (one-sphere and two-sphere Boolean scenarios), we compare the efficiency of the parallel fast sweep method (FSM), the traditional fast iterative method (Tra.FIM), and the fast iterative method combined with multiscale technology (Imp.FIM) under the same accuracy requirement ($RMSE < 0.05$). By varying the number of grid points (computational scale) and testing the algorithm's time consumption, we verify the advantages of the fast iterative method combined with multiscale techniques in large-scale computational domains, and at the same time, we bridge the analytic theory of SDF Boolean operations to explore the algorithm's suitability for complex geometrical representations.

A single sphere distance field with $f(x) = 1$ an initial condition of radius $X = 1.35$ is used as the solution object to test the elapsed time consumption of the parallel Fast Sweep Method (FSM), the traditional Fast Iterative Method (Tra.FIM), and the fast iterative method combined with multiscale technology (Imp.FIM) for different numbers of mesh points (computational scales, e.g., $1E5 - 1E9$ magnitude). The fast iterative method (Imp.FIM) combined with the multiscale technique consistently outperforms the FSM and the traditional FIM, and the advantage becomes more and more significant as the computational scale increases (see Fig. 8 and Table 1 for details). For example, when approaching the scale of 551^3 ($1.67E + 08$), the speed of Imp.FIM is about 2.3 times that of FSM and 24 times that of traditional FIM, which verifies the effectiveness of the multiscale technique in accelerating the global convergence of large-scale 3D monospherical distance fields.

As shown in Fig. 9, for two spheres with centers $(2, 0, 0)$ and $(-2, 0, 0)$ and radii, $X1 = X2 = 1.35$ respectively, based on the analytical definition of the SDF (Symbolic Distance Function) Boolean operation, the intersection, union, and difference operations are executed on the distance field to test the difference between the traditional fast iterative method (Tra.FIM) and the fast iterative method (Tra.FIM) with the combination of multiscale technology (Imp.FIM) at different grid points (e.g., $5.18E+05 - 1.06E+09$). As shown in Table 2, Imp.FIM is about 23.6, 24, and 22 times more efficient than the traditional FIM for intersection, union, and difference operations, respectively, as the number of grid points increases.

5.3. Comparison with FSM and traditional FIM (complex model)

We imported four 3-dimensional complex geometries (dragon, bunny, kitten, and armadillo) as boundaries for signed distance function (SDF) computation. The effectiveness of the solutions is demonstrated through 2D slicing and rendering (Figs. 12). While several algorithms yield similar solution results, their efficiencies differ. In this paper, we present elapsed time tables for various algorithms at different computational scales, using $RMSE < 0.05$ as the accuracy criterion. The results of the Improved Fast Iterative Method (abbreviated as Imp.FIM) are shown here for comparison with the GPU version of the Fast Sweeping Method (FSM) [23] and the traditional FIM (abbreviated as Tra.FIM) [9].

The four 3D complex geometric models of a dragon, rabbit, kitten, and armadillo are imported as the boundaries for the calculation of the signed distance function (SDF), and the root mean square error (RMSE) is less than 0.05 as the accuracy criterion, and we compare the fast scanning method (FSM), the traditional fast iterative method

Table 1

Elapsed time vs. number of grid points. The table presents two sets of data for solving Eq. (1) with $f(x) = 1$ and an initial spherical condition of radius $X = 1.35$. First, it shows the elapsed time (in milliseconds, with $RMSE < 0.05$) for four algorithms – the parallel fast sweeping method, the traditional fast iterative method, and the fast iterative method combined with the multiscale technique – across varying numbers of grid points N . Second, it details the enhancement ratios of the multiscale - integrated fast iterative methods relative to the fast sweeping method and traditional fast iterative method at different computational scales. The enhancement ratio is calculated as (elapsed time of compared method - elapsed time of improved method)/elapsed time of compared method.

Scale	FSM	Tra.FIM	Imp.FIM
1E5	0.7055	2.5945	0.7016
2E6	1.5949	7.3921	1.1348
2E7	11.6371	92.9416	7.2202
2E8	168.7730	1799.7760	73.7563
1E9	3724.9629	16274.8242	518.6007
	vs FSM	vs Tra.FIM	
1E5	0.55%	269.79%	
2E6	40.55%	551.43%	
2E7	61.17%	1187.24%	
2E8	128.83%	2340.17%	
1E9	618.27%	3038.22%	

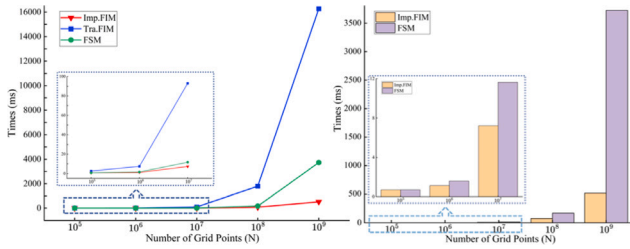


Fig. 8. Elapsed time vs. number of grid points. The figures depict the elapsed time (in milliseconds, with an accuracy of $RMSE < 0.05$) for solving Equation (1) under the condition of $f(x) = 1$ and an initial spherical condition with a radius of $X = 1.35$, plotted against the number of grid points N . The left figure compares four algorithms: the parallel fast sweeping method, the traditional fast iterative method, and the fast iterative method combined with the multiscale technique, while the right histogram focuses on three algorithms, excluding the traditional fast iterative method, to illustrate the time efficiency trends across varying grid resolutions.

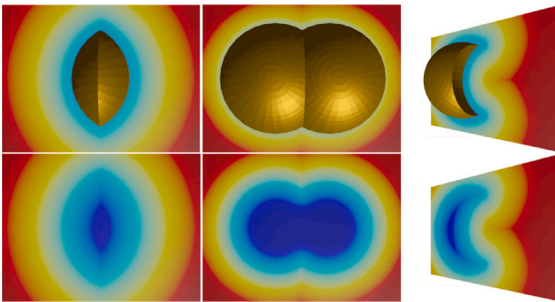


Fig. 9. Plot of the solution for the case $F = 1$ in Eq. (1), where the initial condition is two spheres of radius $X1 = 1.35$ and $X2 = 1.35$, centered at $(2, 0, 0)$ and $(-2, 0, 0)$ respectively. Note that all methods produce exactly the same plots, so only one plot is shown. (Since these algorithms get exactly the same solution, we only show one figure here.)

(Tra.FIM), and the improved fast iterative method (Imp.FIM) under different grid point sizes. The time and efficiency of solving the three-dimensional equation ($f(x) = 1$), the validity of the solution is verified by two-dimensional slicing and rendering, and the difference in the

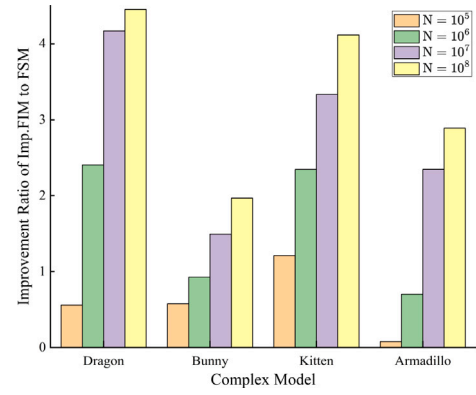


Fig. 10. Improvement Ratio of Imp.FIM to FSM Across 3D Models. This bar chart visualizes the improvement ratio of the Improved Fast Iterative Method (Imp.FIM) relative to the Fast Sweeping Method (FSM) for solving the 3D Eikonal equation ($f(x) = 1$) across four complex 3D models (*dragon*, *bunny*, *kitten*, *armadillo*). Different colors represent grid point scales: $N = 10^5$ (orange), $N = 10^6$ (green), $N = 10^7$ (purple), and $N = 10^8$ (yellow). Higher bars indicate greater efficiency gains of Imp.FIM over FSM at corresponding scales and models.. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

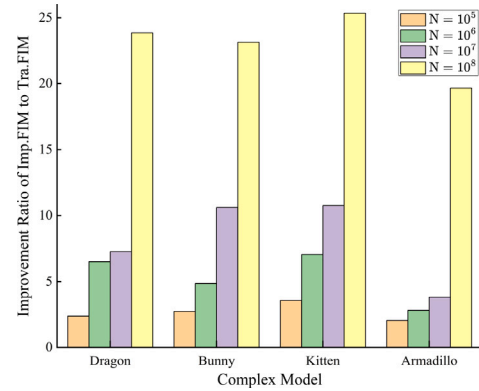


Fig. 11. Improvement Ratio of Imp.FIM to Tra.FIM Across 3D Models. This bar chart visualizes the improvement ratio of the Improved Fast Iterative Method (Imp.FIM) relative to the Tra.FIM for solving the 3D Eikonal equation ($f(x) = 1$) across four complex 3D models (*dragon*, *bunny*, *kitten*, *armadillo*). Different colors represent grid point scales: $N = 10^5$ (orange), $N = 10^6$ (green), $N = 10^7$ (purple), and $N = 10^8$ (yellow). Higher bars indicate greater efficiency gains of Imp.FIM over Tra.FIM at corresponding scales and models. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

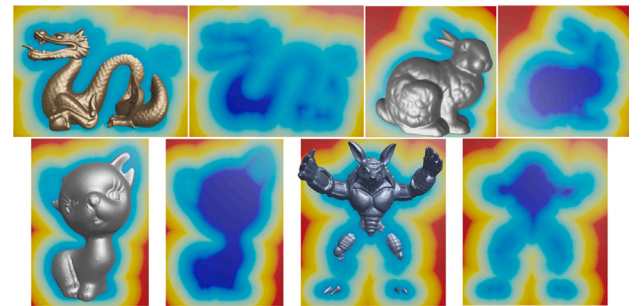


Fig. 12. The figures show the solution of Eq. (1) for $f(x) = 1$, rendered with a distance interval of $[-0.1, 0.5]$. Here the initial conditions are the dragon, bunny, kitten, and armadillo.

Table 2

Elapsed time vs. number of grid points. The table presents set of data for solving Eq. (1) with $f(x) = 1$ and the initial condition is two spheres of radius $X1 = 1.35$ and $X2 = 1.35$, centered at $(2, 0, 0)$ and $(-2, 0, 0)$ respectively. It shows the elapsed time for the traditional fast iterative method, and the fast iterative method combined with the multiscale technique—across varying numbers of grid points N .

Scale	Tra.FIM	Imp.FIM	vs
Intersect			
5.18E+05	1.4356	0.7690	86.68%
5.34E+06	2.7737	0.8458	227.93%
1.48E+07	9.7292	2.5989	274.36%
1.04E+08	216.6036	15.2946	1316.21%
1.06E+09	3786.4431	153.5750	2175.33%
Union			
5.18E+05	1.5360	0.7956	93.05%
5.34E+06	3.0244	0.8550	253.71%
1.48E+07	10.8383	1.6906	541.10%
1.04E+08	237.4161	15.6201	1419.94%
1.06E+09	3494.3369	147.7510	2265.02%
Minus (Difference)			
5.18E+05	1.4490	0.6615	119.04%
5.34E+06	2.7182	0.8253	229.34%
1.48E+07	9.6557	2.2436	330.37%
1.04E+08	231.6289	15.4184	1402.29%
1.06E+09	3471.0420	156.9250	2249.25%

performance of the algorithms is quantified with the help of graphs and tables.

Taking the 3D models of a dragon, rabbit, kitten, and armadillo as the objects, we set the RMSE to < 0.05 precision and tested the elapsed time of FSM, Tra.FIM, and Imp.FIM to solve the SDF with different grid point sizes (e.g., $N = 10^5$ to $N = 10^8$, etc.). Figs. 10 and 11 show the efficiency improvement ratio of Imp.FIM over FSM and Tra.FIM; Tables 5 and 6 present the results of time consumption and speedup ratio of each algorithm.

As the grid point size increases, the efficiency advantage of Imp.FIM over FSM and Tra. Taking the dragon model as an example, Imp.FIM improves the efficiency relative to FSM by 54.21%–240.62%, and relative to Tra.FIM by 238.52%–1088.52%; the rabbit, kitten, and armadillo models show a similar pattern, e.g., the kitten model improves the efficiency relative to FSM by 120.85%–289.03%, and relative to Tra.FIM by 356.32%–1751%.

All three algorithms increase in time consumption with the increase of grid point size, but Imp.FIM increases less. Tables 5 and 6 show that Imp.FIM consumes much less time than FSM and Tra.FIM for large-scale grids, which reflects the speedup effect of the multiscale strategy.

The Imp.FIM algorithm has a significant efficiency improvement compared with FSM and Tra.FIM in SDF computation of complex 3D models such as dragons, rabbits, kittens, armadillos, etc., and its advantage is more prominent with the increase of data size. FIM balances the solution accuracy and computation time through the multiscale technique and verifies the high efficiency of adapting to complex geometrical models and large-scale computational domains in solving 3D programmed function equations, which provides a better choice of algorithms in related geometrical and numerical computation scenarios.

5.4. Algorithm applications

In fluid simulation, efficient algorithms for solving signed distance fields, especially fast iterative methods integrated with multiscale techniques, are essential for achieving quick fluid simulations in intricate scenarios. In multibody systems, particle emitters interact with a plethora of 3D geometries. The varied shapes and complex spatial arrangements of these geometries pose difficulties in accurately computing the signed distances among them.

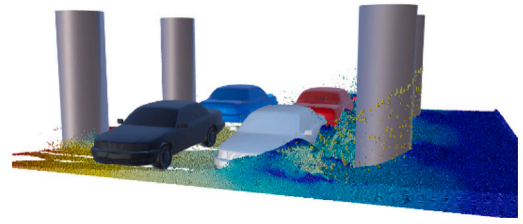


Fig. 13. The simulation of a garage during dam-breaking and subsequent water ingress.

This paper proposes a multiscale-based Fast Iterative Method (FIM) aimed at efficiently computing complex wavefront propagation in multibody systems. To demonstrate its effectiveness, two representative scenarios of complex multibody systems are designed, where the algorithm is utilized to swiftly solve the signed distance field.

5.4.1. A garage scenario during dam-breaking

Fig. 13 depicts a simulation of a garage during a dam break and subsequent water ingress. Four parked cars and multiple columns are visible. On the right, a colored fluid simulation illustrates the post-dam-failure water influx. The water flow's color gradient of green, yellow, and blue reveals its dynamics and distribution as it impacts the vehicles, with debris scattered, evoking a tense and destructive atmosphere. In this scenario, the interaction between the dam-break flow and obstacles like cars and columns generates complex phenomena such as multiple wavefront intersections and strong fluid–structure coupling. These require high-frequency calculations of signed distance fields to capture the dynamic changes at the fluid–solid interface, which precisely matches the algorithm's design goal of handling complex wave propagation.

5.4.2. Flood-fighting scenario

When confronting flood disasters, humans rely on teamwork and their capabilities to actively conduct flood-rescue operations. Fig. 14 depicts baby elephants simulating a human flood-fighting scenario. By virtue of their position distribution and mutual cooperation patterns, they mimic the personnel deployment and cooperation within a human flood-fighting team. Through collective efforts, they attempt to address the rising water level and maintain the relative stability of their environment. This scenario involves real-time calculations of the distance fields between voxels to optimize cooperation paths, presenting high-frequency computational demands similar to practical multibody systems.

The proposed algorithm leverages hierarchical coarse-fine grid acceleration and V-cycle multiscale iterations. It can not only accurately capture the complex effects in wavefront intersection areas, as demonstrated in the garage scenario, but also meet the high-frequency computational requirements of dynamic multibody interactions, as shown in the flood-fighting scenario. These two scenarios, from the perspectives of physical simulation and multibody modeling, comprehensively validate the effectiveness and universality of the algorithm, highlighting its application value in fluid–structure coupling and multibody system simulations. The efficient distance-field solving algorithm introduced in this paper exhibits stable performance across test scenarios. This implies that the developed algorithm can overcome the accuracy degradation and computational inefficiencies commonly associated with traditional methods during large-scale computations. As a result, it offers valuable technical support for subsequent research and practical applications in fluid–structure coupling and related areas.

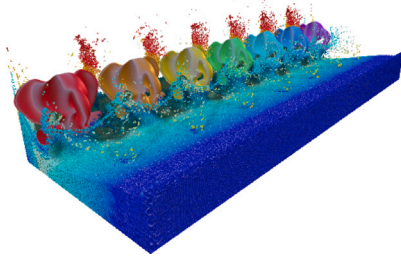


Fig. 14. Flood-fighting scenario. Through collective efforts, elephants, simulating humans, attempt to address the rising water level and maintain the relative stability of their environment.

6. Summary and future work

This paper addresses the challenging problem of high-frequency signed distance computation for horizontal sets, particularly in large-scale scenarios, and conducts an in-depth investigation into the optimization of the Fast Iterative Method (FIM) algorithms through the integration of parallelism and multiscale technology.

In the introduction, we elucidate the limitations of traditional computational methods in managing large-scale and high-frequency directed distance computations, including low computational efficiency and excessive resource consumption. This discussion clearly underscores the urgent need to incorporate two cutting-edge technologies – parallelism and multiscale techniques – into the FIM algorithm.

Subsequently, we construct various scales of grid hierarchies utilizing multiscale technology, which are effectively integrated with FIM to leverage the efficient computational properties of coarse grids, thereby optimizing the overall computational process.

The convergence of the proposed algorithm is experimentally validated, and a series of comparative experiments are designed. In the experimental setup, all algorithms are benchmarked against the GPU version. The results indicate that, compared to the GPU version of the FSM algorithm, the improved FIM algorithm achieves approximately a 130% increase in computational efficiency, and it demonstrates a remarkable 22-fold efficiency improvement over the traditional FIM algorithm. Furthermore, the algorithm is applied to the simulation of multibody systems, where it operates stably in modeling complex processes such as fluid–structure coupling.

We present the following findings and reflections from the research process to inspire future work:

Findings and reflections.

- (1) A small number of FIM iterations must be computed on the finest mesh before entering the V-loop to ensure that as much low-frequency information as possible is available for iterative updates on the coarser mesh. However, this approach inevitably increases time overhead. It is therefore advisable to further investigate the relationship between low-frequency information (fine-mesh iterations) and high-frequency updates (coarse-mesh iterations).
- (2) Typically, following the initialization of the computational domain, a single iteration of the V-loop computation ensures that all grid points initially assigned infinite states are updated to tentative states. This transformation eliminates the need for subsequent V-loop iterations to identify and process grid points with infinite states and their neighboring points, thereby significantly reducing computational overhead.

Additionally, the algorithm can be applied to related research efforts in various simulation scenarios. For instance, in the simulations of floods, earthquakes, and other complex real-world scenarios, it can rapidly and accurately model the evolution of disasters, thereby providing valuable support for the development of disaster warning and response strategies.

Table 3

Shows the RMSE values corresponding to the iterative process of the V-cycle for the four scales. Here are the calculations for the scales 64^3 , 155^3 , 280^3 , and 560^3 , respectively, for $f(x) = 1$ in Eq. (1), where the initial condition is a sphere of radius $X = 1.35$.

N	64^3	155^3	280^3	560^3
1	48.4541	6.8514	14.7342	9.6317
2	25.6290	2.7832	9.4381	6.5937
3	10.7048	0.3220	4.7840	3.4188
4	1.9513	0.0297	2.1584	1.5911
5	0.0486	0.0288	0.2140	0.3539
6	0.0471	0.0281	0.0181	0.0788
7	0.0460	0.0276	0.0176	0.0261
8	0.0452	0.0271	0.0172	0.0257
9	0.0446	0.0267	0.0168	0.0254
10	0.0442	0.0264	0.0164	0.0251

Table 4

The table shows the RMSE values corresponding to different numbers of V-cycles at a 64^3 scale for initial values of 500, 1000, 2000, and 3000, respectively.

	IV 500	IV 1000	IV 2000	IV 3000
1	24.4355	48.8666	97.3093	146.0377
2	12.9984	26.1185	51.5458	77.2163
3	5.2801	10.7608	20.9527	30.8691
4	0.97603	1.8917	3.9041	5.8571
5	0.0581	0.0486	0.0486	0.0485
6	0.0557	0.0471	0.0471	0.0470
7	0.0535	0.0461	0.0461	0.0460
8	0.0516	0.0453	0.0453	0.0452
9	0.0499	0.0447	0.0447	0.0446
10	0.0484	0.0442	0.0442	0.0442

CRediT authorship contribution statement

Jingqi Zhang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Zihao Zhou:** Conceptualization, Data curation, Formal analysis, Methodology, Resources, Software, Validation, Visualization, Writing – review & editing. **Lixin Ren:** Conceptualization, Methodology, Software, Validation, Visualization, Writing – review & editing. **Junyuan Liu:** Conceptualization. **Ying Li:** Funding acquisition, Project administration, Supervision. **Xiaowei He:** Funding acquisition, Methodology, Project administration, Software, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the anonymous reviewers for their constructive comments. This research was partially supported by funding from the Ministry of Industry and Information Technology of the People's Republic of China (MIIT) Industrial Real-Time Database Project Based on New-Generation Information Technology Project (No.TC210804D) and from the Institute of Software, Chinese Academy of Sciences (ISCAS) Basic Research Project (No.ISCAS-JCMS-202403).

Table 5

Elapsed Time and Speedup for 3D Eikonal Equation Solvers (Imp.FIM vs. Tra.FSM). The table presents elapsed times (in milliseconds, RMSE < 0.05) for solving the 3D Eikonal equation ($f(x) = 1$) with initial conditions shaped as 3D models (*dragon*, *kitten*, *bunny*, *armadillo*). Two methods are compared across grid point scales (N): the Traditional Fast Iterative Method (Tra.FIM) and the Improved Fast Iterative Method with multiscale (Imp.FIM). The speedup ratio ("vs Tra.FIM") is calculated as $\frac{\text{Tra.FIM Elapsed Time} - \text{Imp.FIM Elapsed Time}}{\text{Imp.FIM Elapsed Time}} \times 100\%$ directly quantifying the efficiency gain of Imp.FIM relative to Tra.FIM for each computational scale.

Scale	Tra.FIM	Imp.FIM	vs Tra.FIM
dragon			
5.85E+05	5.1086	1.5091	238.52%
5.40E+06	86.6818	11.5578	649.99%
5.06E+07	1514.3929	183.6392	724.66%
5.07E+08	34779.4766	1398.3105	2387.25%
kitten			
8.26E+05	7.2840	1.5963	356.32%
8.16E+06	142.5283	17.7106	704.76%
5.23E+07	1679.8774	143.0935	1073.97%
5.14E+08	33126.2969	1257.1639	2535.00%
bunny			
4.75E+05	4.7962	1.2837	273.63%
4.26E+06	75.9862	12.9672	485.99%
2.79E+07	854.3136	73.6844	1059.42%
4.33E+08	30992.2227	1282.5790	2316.40%
armadillo			
9.78E+05	9.6405	2.9009	204.82%
9.60E+06	172.9909	30.7020	281.57%
6.96E+07	2281.7598	184.6683	381.61%
5.14E+08	32233.0488	1560.3039	1965.82%

Table 6

Elapsed Time and Speedup for 3D Eikonal Equation Solvers (Imp.FIM vs. FSM).

Scale	FSM	Imp.FIM	vs FSM
dragon			
5.85E+05	2.3532	1.5091	55.93%
5.40E+06	39.3685	11.5578	240.62%
5.06E+07	949.1524	183.6392	416.86%
5.07E+08	7626.7871	1398.3105	445.43%
1.01E+09	11712.1709	1512.6639	674.27%
kitten			
8.26E+05	3.5254	1.5963	120.85%
8.16E+06	59.2673	17.7106	234.64%
5.23E+07	619.6002	143.0935	333.00%
5.14E+08	6431.6797	1257.1639	411.60%
1.07E+09	27389.5703	1662.4751	1547.52%
bunny			
4.75E+05	2.0224	1.2837	57.55%
4.26E+06	25.0146	12.9672	92.91%
2.79E+07	183.5373	73.6844	149.09%
4.33E+08	3808.1499	1282.5790	196.91%
1.03E+09	6542.2510	1254.6142	421.46%
armadillo			
9.78E+05	3.7130	2.9009	7.69%
9.60E+06	64.3512	30.7020	70.44%
6.96E+07	470.4139	184.6683	234.78%
5.14E+08	6070.0352	1560.3039	289.03%
1.05E+09	9659.6367	1438.9657	571.29%

Appendix A

This section includes supplementary materials to support the main text. Table 3, Table 4, Table 5, and Table 6 present additional RMSE values, elapsed time, and speedup data for 3D Eikonal equation solvers.

These tables provide further insights into the performance of different models (like dragon, kitten, bunny, and armadillo) across various scales, complementing the analysis of iterative processes and computational efficiency in the main content.

Appendix B. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cad.2025.103949>.

Data availability

No data was used for the research described in the article.

References

- [1] Sui H, An K, Xu C, Liu J, Feng W. Flood detection in PolSAR images based on level set method considering prior geoinformation. *IEEE Geosci Remote Sens Lett* 2018-05;15(5):699–703. <http://dx.doi.org/10.1109/LGRS.2018.2810122>, URL <http://ieeexplore.ieee.org/document/8315453/>.
- [2] Enright D, Losasso F, Fedkiw R. A fast and accurate semi-lagrangian particle level set method. *Comput Struct* 2005-02;83(6):479–90. <http://dx.doi.org/10.1016/j.compstruc.2004.04.024>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0045794904004195>.
- [3] Kumar R, Lopez O, Davis D, Aravkin AY, Herrmann FJ. Beating level-set methods for 5-d seismic data interpolation: a primal-dual alternating approach. *IEEE Trans Comput Imaging* 2017-06;3(2):264–74. <http://dx.doi.org/10.1109/TCI.2017.2693966>, URL <https://ieeexplore.ieee.org/document/7906537/>.
- [4] Espinosa CB, Sánchez J, Appel S, Becker S, Krauß J, Díaz PM, Unger L, Houillon M, Loewe A. A cyclical fast iterative method for simulating reentries in cardiac electrophysiology using an eikonal-based model. 2024-06-26, <http://dx.doi.org/10.48550/arXiv.2406.18619>, arXiv:2406.18619[physics].
- [5] Gibou F, Fedkiw R, Osher S. A review of level-set methods and some recent applications. *J Comput Phys* 2018-01;353:82–109. <http://dx.doi.org/10.1016/j.jcp.2017.10.006>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0021999117307441>.
- [6] Weinbub J, Hössinger A. Accelerated redistancing for level set-based process simulations with the fast iterative method. *J Comput Electron* 2014-12;13(4):877–84. <http://dx.doi.org/10.1007/s10825-014-0604-x>, URL <http://link.springer.com/10.1007/s10825-014-0604-x>.
- [7] Capozzoli A, Curcio C, Lisenio A, Savarese S. A comparison of fast marching, fast sweeping and fast iterative methods for the solution of the eikonal equation. In: 2013 21st telecommunications forum telfor (TELFOR). IEEE; 2013-11, p. 685–8. <http://dx.doi.org/10.1109/TELFOR.2013.6716321>, URL <http://ieeexplore.ieee.org/document/6716321/>.
- [8] Cai W, Zhu P, Li G. Improved fast iterative method for higher calculation accuracy of traveltimes. *Comput Geosci* 2023-05;174:105331. <http://dx.doi.org/10.1016/j.cageo.2023.105331>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0098300423000353>.
- [9] Huang Y. Improved fast iterative algorithm for eikonal equation for GPU computing. 2021-08-02, <http://dx.doi.org/10.48550/arXiv.2106.15869>, arXiv:2106.15869[math].
- [10] Quell M, Diamantopoulos G, Hössinger A, Selberherr S, Weinbub J. Parallel correction for hierarchical re-distancing using the fast marching method. In: Dimov I, Fidanova S, editors. *Advances in high performance computing*. vol. 902, Springer International Publishing; 2021, p. 438–51. http://dx.doi.org/10.1007/978-3-030-55347-0_37, Series Title: *Studies in Computational Intelligence* URL http://link.springer.com/10.1007/978-3-030-55347-0_37.
- [11] Shrestha A, Senocak I. Multi-level domain-decomposition strategy for solving the Eikonal equation with the fast-sweeping method. *IEEE Trans Parallel Distrib Syst* 2018-10-01;29(10):2297–303. <http://dx.doi.org/10.1109/TPDS.2018.2829869>, URL <https://ieeexplore.ieee.org/document/8345737/>.
- [12] Dang F, Emad N. Fast iterative method in solving Eikonal equations: A multi-level parallel approach. *Procedia Comput Sci* 2014;29:1859–69. <http://dx.doi.org/10.1016/j.procs.2014.05.170>, URL <https://linkinghub.elsevier.com/retrieve/pii/S1877050914003470>.
- [13] Gomez JV, Alvarez D, Garrido S, Moreno L. Fast methods for Eikonal equations: An experimental survey. *IEEE Access* 2019;7:39005–29. <http://dx.doi.org/10.1109/ACCESS.2019.2906782>, URL <https://ieeexplore.ieee.org/document/8672778/>.
- [14] Fu Z, Kirby RM, Whitaker RT. A fast iterative method for solving the Eikonal equation on tetrahedral domains. *SIAM J Sci Comput* 2013-01;35(5):C473–94. <http://dx.doi.org/10.1137/120881956>, URL <http://epubs.siam.org/doi/10.1137/120881956>.

- [15] Chen W, Chou C-S, Kao C-Y. Lax–Friedrichs multigrid fast sweeping methods for steady state problems for hyperbolic conservation laws. *J Sci Comput* 2015-09;64(3):591–618. <http://dx.doi.org/10.1007/s10915-015-0006-7>, URL <http://link.springer.com/10.1007/s10915-015-0006-7>.
- [16] Sethian JA. A fast marching level set method for monotonically advancing fronts.. *Proceedings of the National Academy of Sciences* 1996;93(4):1591–5. <http://dx.doi.org/10.1073/pnas.93.4.1591>, URL <https://www.pnas.org/doi/10.1073/pnas.93.4.1591>.
- [17] Diamantopoulos G, Hössinger A, Selberherr S, Weinbub J. A shared memory parallel multi-mesh fast marching method for re-distancing. *Adv Comput Math* 2019-08;45(4):2029–45. <http://dx.doi.org/10.1007/s10444-019-09683-z>, URL <http://link.springer.com/10.1007/s10444-019-09683-z>.
- [18] Yang J, Stern F. A highly scalable massively parallel fast marching method for the Eikonal equation. *J Comput Phys* 2017-03;332:333–62. <http://dx.doi.org/10.1016/j.jcp.2016.12.012>, arXiv:1502.07303[physics].
- [19] Chacon A, Vladimirovsky A. Fast two-scale methods for eikonal equations. *SIAM J Sci Comput* 2012-01;34(2):A547–78. <http://dx.doi.org/10.1137/10080909X>, URL <http://epubs.siam.org/doi/10.1137/10080909X>.
- [20] Breuß M, Cristiani E, Gwosdek P, Vogel O. An adaptive domain-decomposition technique for parallelization of the fast marching method. *Appl Math Comput* 2011-09;218(1):32–44. <http://dx.doi.org/10.1016/j.amc.2011.05.041>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0096300311007107>.
- [21] Zhao H. A fast sweeping method for eikonal equations. *Mathematics of Computation* 2005;74(250):603–27. <http://dx.doi.org/10.1090/S0025-5718-04-01678-3>, URL <https://www.ams.org/journals/mcom/2005-74-250/S0025-5718-04-01678-3/>.
- [22] Detrixhe M, Gibou F, Min C. A parallel fast sweeping method for the eikonal equation. *J Comput Phys* 2013-03;237:46–55. <http://dx.doi.org/10.1016/j.jcp.2012.11.042>, URL <https://linkinghub.elsevier.com/retrieve/pii/S002199911200722X>.
- [23] Zhao H. Parallel implementations of the fast sweeping method. *Journal of Computational Mathematics* 2007;25(3):421–9. <http://dx.doi.org/10.4208/jcm.0701-m2006-0118>, URL <https://www.global-sci.org/jcm/article/25/3/421.html>.
- [24] Jeong W-K, Whitaker RT. A fast iterative method for eikonal equations. *SIAM J Sci Comput* 2008-01;30(5):2512–34. <http://dx.doi.org/10.1137/060670298>, URL <http://epubs.siam.org/doi/10.1137/060670298>.
- [25] Hong S, Jeong W-K. A multi-gpu fast iterative method for eikonal equations using on-the-fly adaptive domain decomposition. *Procedia Computer Science* 2016;80:190–200. <http://dx.doi.org/10.1016/j.procs.2016.05.456>, URL <https://linkinghub.elsevier.com/retrieve/pii/S1877050916304563>.
- [26] Hong S, Jeong W-K. A group-ordered fast iterative method for eikonal equations. *IEEE Transactions on Parallel and Distributed Systems* 2016;28(2):318–31. <http://dx.doi.org/10.1109/TPDS.2016.2598364>, URL <https://ieeexplore.ieee.org/document/7552444/>.
- [27] Detrixhe M, Gibou F. Hybrid massively parallel fast sweeping method for static hamilton-jacobi equations. *J Comput Phys* 2016-10;322:199–223. <http://dx.doi.org/10.1016/j.jcp.2016.06.023>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0021999116302480>.